

1 Variational Autoencoder (VAE) Derivations

1.1 Reconstruction Loss

- a) In the code, the decoder models $p_\theta(x | z) = \mathcal{N}(f_\theta(z), \sigma^2 I)$, where $f_\theta(z)$ is the decoder output and σ^2 is a fixed constant.

The formula for the normal multivariate gaussian is as follows,

$$\begin{aligned} p_\theta(x | z) &= \frac{1}{(2\pi)^{D/2} |\sigma^2 I|^{1/2}} \exp \left(-\frac{1}{2} (x - f_\theta(z))^\top (\sigma^2 I)^{-1} (x - f_\theta(z)) \right) \\ &= \frac{1}{(2\pi\sigma^2)^{D/2}} \exp \left(-\frac{1}{2\sigma^2} \|x - f_\theta(z)\|^2 \right) \end{aligned}$$

Taking log of the decoder probability we get,

$$\log p_\theta(x | z) = -\frac{1}{2\sigma^2} \|x - f_\theta(z)\|^2 - \frac{D}{2} \log(2\pi\sigma^2)$$

If we take the σ^2 as a fixed constant then the second term becomes a fixed constant.

$$\log p_\theta(x | z) = -\frac{1}{2\sigma^2} \|x - f_\theta(z)\|^2 + \text{constant}$$

Now the decoder wants to maximize the log likelihood of $\log p_\theta(x | z)$. From the above equation it is quite clear that maximizing the log likelihood $\log p_\theta(x | z)$ is equivalent to minimizing $\|x - f_\theta(z)\|^2$ which is exactly equal to the Mean Squared Error (MSE) between the original image x and the reconstructed image $f_\theta(z)$.

- b) Here, we have implemented a deterministic decoder that outputs only the mean of the distribution, $f_\theta(z)$, while ignoring the variance. As a result, the decoder cannot capture uncertainty or the multimodal structure of the data. When multiple plausible outputs correspond to the same latent variable z —for example, different images of the same class—the model effectively averages these possibilities. ***This averaging leads to blurry generated images.*** While VAEs already tend to produce blurry images due to the use of MSE loss, employing a deterministic decoder further exacerbates this issue by introducing additional smoothing.

When the decoder is deterministic, the VAE loses its ability to inject stochasticity at the output stage. Consequently, ***all uncertainty and variation must be captured alone in the latent space.*** This reduces outputs diversity, as multiple passes of the same latent code z will always produce identical images. Therefore, ***all diversity in generation must arise from sampling different latent codes, rather than from any randomness in the decoder.***



(a) VAE Output (Blurry)



(b) Ground Truth / Sharper Image

Figure 1: Comparison showing the blurriness in VAE-generated images due to deterministic decoding.

- c) If we model each pixel as an independent bernoulli: $p_{\theta}(x | z) = \prod_{i=1}^D \text{Bern}(x_i | \hat{x}_i)$ where D is the number of pixels.

We want to learn $\hat{x}_i$ for each pixel having $p_{\theta}(x_i | z) = \text{Bernoulli}(f_{\theta}(z))$.

Taking log of the probability we get,

$$\log p_{\theta}(x | z) = \sum_i [x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i)]$$

This corresponds to the binary cross-entropy loss.

Now modeling each pixel as Bernoulli is much more principled than Gaussian because while Gaussian encourages averaging which translates to blurring due to the Mean Squared Error(MSE), while Bernoulli predicts the probability of 1 for each pixel in the image which is particularly appropriate for MNIST since the pixels in the dataset are either black or white (0 or 1). Thus the ***binary cross entropy error(BCE) thus matches the support of the MNIST pixels exactly as BCE assumes $x \in \{0, 1\}$ whereas MSE allows unbounded real values.*** Thus independent Bernoulli distribution for each pixel produces much **sharper images** than that of Gaussian or Normal distribution.

Another problem under MSE is that predicting 0.5 for a pixel that is 1 yields the same penalty as predicting 0.5 for a pixel that is 0. However, from a probabilistic standpoint, both cases correspond to maximally uncertain Bernoulli predictions. Binary cross-entropy (BCE) correctly treats this as maximum entropy and penalizes it accordingly.

1.2 Closed-Form KL Divergence

Given, $q_\theta(z | x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and $p(z) = \mathcal{N}(0, I)$. Let us start with the definition of KL divergence

$$\begin{aligned} D_{\text{KL}}(q_\theta(z | x) \| p(z)) &= \int q_\theta(z | x) \log \frac{q_\theta(z | x)}{p(z)} dz \\ &= \mathbb{E}_{q_\theta(z|x)} \left[\log \frac{q_\theta(z | x)}{p(z)} \right] \\ &= \mathbb{E}_{q_\theta(z|x)} [\log q_\theta(z | x) - \log p(z)] \end{aligned} \quad (1)$$

Now let us expand the 2 terms within the expectation separately.

$$\log q_\theta(z | x) = -\frac{1}{2} \sum_{j=1}^d \left(\log(2\pi\sigma_j^2) + \frac{(z_j - \mu_j)^2}{\sigma_j^2} \right), \quad (2)$$

$$\log p(z) = -\frac{1}{2} \sum_{j=1}^d (\log(2\pi) + z_j^2). \quad (3)$$

Now subtracting (3) from (2) we get

$$\begin{aligned} \log q_\theta(z | x) - \log p(z) &= -\frac{1}{2} \sum_{j=1}^d \left(\log(2\pi\sigma_j^2) + \frac{(z_j - \mu_j)^2}{\sigma_j^2} \right) + \frac{1}{2} \sum_{j=1}^d (\log(2\pi) + z_j^2) \\ &= -\frac{1}{2} \sum_{j=1}^d \left(\log \sigma_j^2 + \frac{(z_j - \mu_j)^2}{\sigma_j^2} - z_j^2 \right) \\ &= -\frac{1}{2} \sum_{j=1}^d \left(\log \sigma_j^2 + \frac{z_j^2 - 2\mu_j z_j + \mu_j^2}{\sigma_j^2} - z_j^2 \right) \end{aligned} \quad (4)$$

Now let us define the expectation of z_j^2 with respect to $q_\theta(z|x)$

$$\mathbb{E}_{q_\theta(z|x)}[z_j^2] = \mu_j^2 + \sigma_j^2$$

This comes from the simple definition of variance.

Now we can substitute equation (4) back in equation (1),

$$\begin{aligned}
D_{\text{KL}}(q_\theta(z \mid x) \parallel p(z)) &= \mathbb{E}_{q_\theta(z \mid x)} \left[-\frac{1}{2} \sum_{j=1}^d \left(\log \sigma_j^2 + \frac{z_j^2 - 2\mu_j z_j + \mu_j^2}{\sigma_j^2} - z_j^2 \right) \right] \\
&= -\frac{1}{2} \sum_{j=1}^d \mathbb{E}_{q_\theta(z \mid x)} \left[\log \sigma_j^2 + \frac{z_j^2 - 2\mu_j z_j + \mu_j^2}{\sigma_j^2} - z_j^2 \right] \\
&= -\frac{1}{2} \sum_{j=1}^d \left(\log \sigma_j^2 + \frac{\mathbb{E}[z_j^2] - 2\mu_j \mathbb{E}[z_j] + \mu_j^2}{\sigma_j^2} - \mathbb{E}[z_j^2] \right) \\
&= -\frac{1}{2} \sum_{j=1}^d \left(\log \sigma_j^2 + \frac{(\mu_j^2 + \sigma_j^2) - 2\mu_j^2 + \mu_j^2}{\sigma_j^2} - (\mu_j^2 + \sigma_j^2) \right) \\
&= -\frac{1}{2} \sum_{j=1}^d \left(\log \sigma_j^2 + \frac{\sigma_j^2}{\sigma_j^2} - \mu_j^2 - \sigma_j^2 \right) \\
&= -\frac{1}{2} \sum_{j=1}^d (\log \sigma_j^2 + 1 - \mu_j^2 - \sigma_j^2)
\end{aligned}$$

This is the final closed form expression of the KL divergence term in VAE.

2 Implementation

2.1 Reparametrization Trick

Why is the reparameterization trick necessary? In a VAE, we need to backpropagate gradients through the sampling operation $z \sim \mathcal{N}(\mu, \sigma^2)$ to the encoder parameters that produce μ and σ . However, direct sampling introduces a stochastic node with no deterministic dependence on μ and σ , making it non-differentiable for gradient-based optimization.

If we directly call `.sample()`, it generates samples that are not connected to the computation graph that pytorch maintains for back propagation. As a result, gradients $\frac{\partial z}{\partial \mu}$ and $\frac{\partial z}{\partial \sigma}$ cannot be computed, and backpropagation stops at z . So the encoder receives no gradient signal from the reconstruction term in the loss and cannot learn from the data. Note the gradient updates from the KL divergence term in the loss function still flows through the encoder.

How reparameterization fixes this ? Reparameterization used to propagate the derivative of ϕ inside the expectation term of Variational Inference(VI) is also used here. The reparameterization trick rewrites the sampling process as:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Here, ϵ introduces randomness independently of the model parameters, while z becomes a deterministic function of μ and σ . This allows gradients to flow through z :

$$\frac{\partial z}{\partial \mu} = 1, \quad \frac{\partial z}{\partial \sigma} = \epsilon.$$

The reparameterization trick separates stochasticity from the parameters, allowing gradients to propagate through the sampling process and enabling the encoder to be trained using backpropagation.

2.2 Loss Function

The loss terms obtained in Section 1 is used here and implemented in the ipynb. Here is the code snippet.

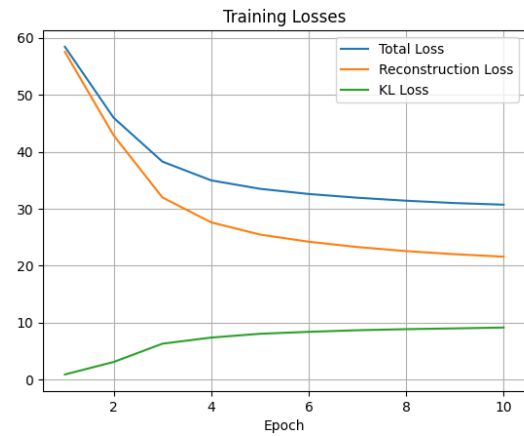
```
def vae_loss(recon_x, x, mu, logvar):
    B = x.size(0)
    recon_x = recon_x.view(B, -1)
    x = x.view(B, -1)
    recon_l = F.mse_loss(recon_x, x, reduction='none')
    recon_l = recon_l.sum(dim=1).mean()
    kl_l = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp())
    return recon_l, kl_l
```

2.3 Training Loop

I have implemented the code in the ipynb. Here are some of the outputs asked in the question. I tested with different number of epochs and reached the conclusion that after 10 epochs further training only showed marginal gains so all the below results are based on training the models for 10 epochs. I only changed the batch size from 128 to 512 to speed up my training iteration.

Epoch	Total Loss	Reconstruction Loss	KL Loss
1	58.426426	57.533244	0.893183
2	45.958838	42.874073	3.084766
3	38.264286	31.971499	6.292788
4	34.970983	27.601231	7.369753
5	33.493524	25.458220	8.035305
6	32.580513	24.201941	8.378571
7	31.925738	23.266727	8.659011
8	31.397594	22.551350	8.846244
9	30.988611	22.006146	8.982465
10	30.703770	21.571326	9.132443

(a) Log of all epochs separately



(b) Graph showing all the three losses

Figure 2: Sample output of the training code

2.4 Monte Carlo KL Approximation

1. Here is the code snippet implemented for Monte Carlo KL Approximate

```
from torch.distributions import Normal
def vae_loss_mc(recon_x, x, mu, logvar, L=1):
    B = x.size(0)
    recon_x = recon_x.view(B, -1)
    x = x.view(B, -1)
    recon_l = F.mse_loss(recon_x, x, reduction='none')
    recon_l = recon_l.sum(dim=1).mean()
    std = torch.exp(0.5 * logvar)
    q = Normal(mu, std)
    z = q.rsample((L,))
    log_q = q.log_prob(z).sum(-1)
    p = Normal(0, 1)
    log_p = p.log_prob(z).sum(-1)
    kl_l = (log_q - log_p).mean(dim=0).mean()
    return recon_l, kl_l
```

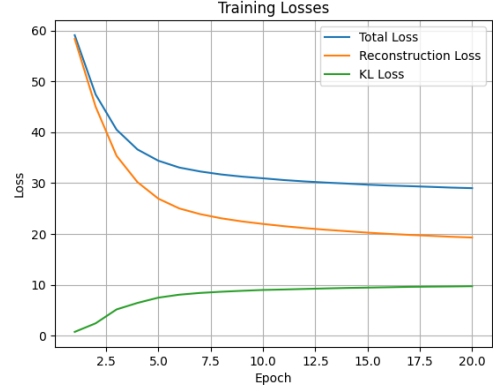
2. This section includes the sample outputs the three different values of $L = \{1, 10, 100\}$

$L = 1$

Final Losses after 20 epochs:
Total Loss: 28.9933
Reconstruction Loss: 19.2828
KL Loss: 9.7105

Epoch	Total Loss	Reconstruction Loss	KL Loss
1	59.066074	58.338104	0.727970
2	47.410619	44.995593	2.415026
3	40.493438	35.349180	5.144257
4	36.599016	30.175297	6.423719
5	34.380169	26.918202	7.461967
6	33.044718	24.999879	8.044838
7	32.268232	23.880138	8.388094
8	31.686152	23.065198	8.620954
9	31.256388	22.449639	8.806750
10	30.926187	21.953387	8.972799
11	30.580799	21.515648	9.065150
12	30.312239	21.138235	9.174003
13	30.088699	20.813805	9.274895
14	29.894087	20.526026	9.368062
15	29.674666	20.239653	9.435013
16	29.507477	20.002065	9.505413
17	29.387831	19.802939	9.584892
18	29.247415	19.620921	9.626494
19	29.098767	19.435435	9.663333
20	28.993276	19.282769	9.710508

(a) Logs for $L=1$



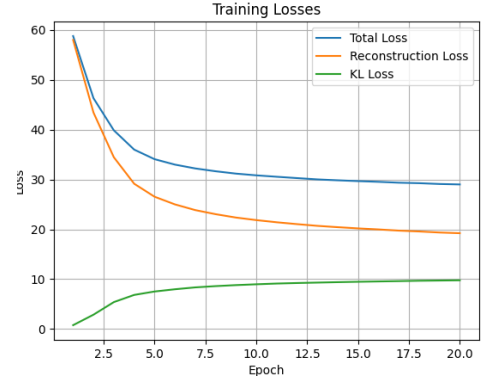
(b) Graph for $L=1$

$L = 10$

Final Losses after 20 epochs:
Total Loss: 29.0044
Reconstruction Loss: 19.2251
KL Loss: 9.7793

Epoch	Total Loss	Reconstruction Loss	KL Loss
1	58.766527	57.989260	0.777267
2	46.307321	43.428296	2.879025
3	39.862427	34.458689	5.403738
4	35.991212	29.145969	6.845243
5	34.073437	26.543952	7.529486
6	32.983842	25.005330	7.978513
7	32.216173	23.860310	8.355863
8	31.649018	23.039522	8.609496
9	31.177538	22.363667	8.813871
10	30.830104	21.851160	8.978945
11	30.545347	21.418786	9.126561
12	30.276825	21.045529	9.231296
13	30.018502	20.699892	9.318610
14	29.835657	20.431435	9.404222
15	29.673363	20.190157	9.483206
16	29.531056	19.981896	9.549160
17	29.355448	19.743031	9.612417
18	29.260402	19.572975	9.687427
19	29.096917	19.365009	9.731908
20	29.004369	19.225100	9.779269

(c) Logs for $L=10$



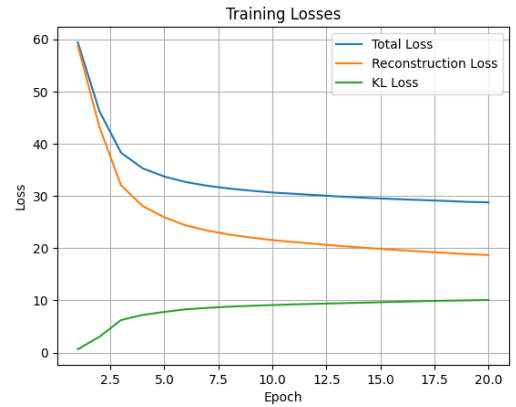
(d) Graph for $L=10$

$L = 100$

Final Losses after 20 epochs:
Total Loss: 28.7720
Reconstruction Loss: 18.6835
KL Loss: 10.0885

Epoch	Total Loss	Reconstruction Loss	KL Loss
1	59.431702	58.785736	0.645966
2	46.224564	43.188492	3.036072
3	38.278960	32.034235	6.244725
4	35.284129	28.068234	7.215894
5	33.715431	25.916197	7.799235
6	32.661945	24.360094	8.301852
7	31.938815	23.370161	8.568654
8	31.413849	22.607158	8.806691
9	31.011506	22.046601	8.964905
10	30.651016	21.544425	9.106592
11	30.416789	21.169465	9.247324
12	30.168090	20.823355	9.344735
13	29.917159	20.471428	9.445731
14	29.709992	20.155298	9.554694
15	29.517645	19.861076	9.656569
16	29.351042	19.594111	9.756930
17	29.200236	19.336843	9.863393
18	29.052687	19.102215	9.950472
19	28.871502	18.867365	10.004137
20	28.772043	18.683511	10.088532

(e) Logs for $L=100$



(f) Graph for $L=100$

Figure 3: Comparison of VAE outputs for different numbers of Monte Carlo samples L .

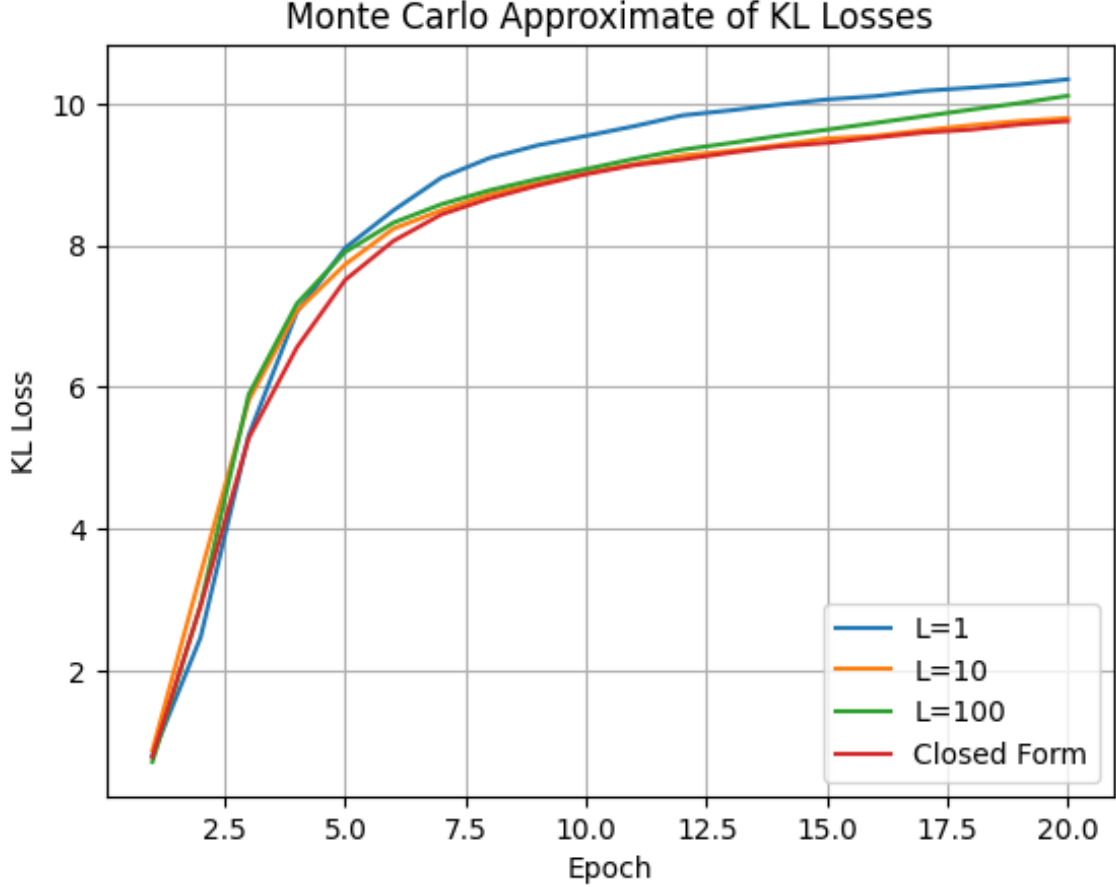


Figure 4: Graph Showing KL loss for different L values against the Closed Form

3. Analysis of KL Loss Curves: MC Sampling vs Closed Form

Comparison of $L = 1$ and $L = 100$ KL Curves. For $L = 1$, the KL curve is much **noisier and more erratic**, as the high variance in the Monte Carlo estimate leads to unstable gradient updates. The KL term increases more **slowly during early epochs**, since noisy gradients delay the learning of a well-structured latent space. The curve may also exhibit **spikes and dips** that do not reflect the true KL value.

In contrast, for $L = 100$, the KL curve is significantly **smoother and more stable**. The KL term increases in a more **consistent and predictable** way and the model converges to a better estimate of the actual KL divergence. This indicates that increasing L will give a better expectation estimate at the cost of training time.

Effect of High Variance in Early Training. The early stage of training is particularly sensitive, as both the encoder and decoder are close to random initialization. High-variance gradient estimates during this phase are harmful for many reasons. The encoder has not yet learned a meaningful posterior, so noisy KL gradients push it in **conflicting directions** across batches. Additionally, the reconstruction loss is also large at the beginning, and the interaction of two noisy objectives may lead to **unstable optimization**. Poor latent representations learned early can be **difficult to recover from**, as the decoder

adapts to a suboptimal latent structure. Finally, with $L = 1$, the gradient estimate may even have the **wrong sign**, resulting in updates that worsen the objective.

Why Closed-Form KL is Preferred. For Gaussian distributions, the KL divergence admits a closed-form expression:

$$D_{\text{KL}}(q \parallel p) = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1).$$

Even with a large number of Monte Carlo samples, the closed-form expression is preferable. It provides **zero-variance** and **exact gradients**, whereas Monte Carlo estimates always retain some residual noise and are only approximate. Also it is **computationally efficient**, requiring only a single forward pass, and is **numerically stable** by construction. Using Monte Carlo estimation always produces a less efficient and yields inferior gradient quality if the closed form expression is available.

Student Name: Arghyadeep Saha

Roll Number: 251110014

Date: May 2, 2026

3 β -VAE and Posterior Collapse

3.1 Training with different β

This section presents the sample outputs for three different values of $\beta = \{0.1, 1, 10\}$. It can be observed that as β increases, the reconstruction loss also increases. This is because a higher weight on the KL divergence term encourages the approximate posterior $q_\phi(z | x)$ to be closer to the prior $p(z)$, therefore restricting the amount of information that can be encoded in the latent variable z .

$\beta = 0.1$

Final Losses after 10 epochs:

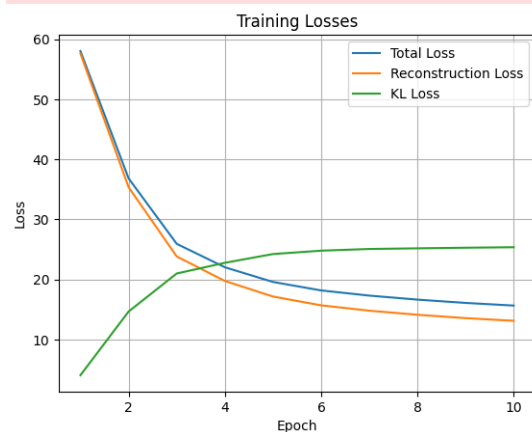
Total Loss: 15.6755

Reconstruction Loss: 13.1373

KL Loss: 25.3825

Epoch	Total Loss	Reconstruction Loss	KL Loss
1	58.027169	57.618711	4.084578
2	36.844632	35.376793	14.678388
3	25.946719	23.845583	21.011356
4	22.048922	19.770064	22.788579
5	19.603238	17.178313	24.249246
6	18.191128	15.710201	24.809278
7	17.330902	14.821728	25.091738
8	16.664602	14.144733	25.198691
9	16.114730	13.584660	25.300699
10	15.675508	13.137263	25.382453

(a) Logs for $\beta = 0.1$



(b) Graph for $\beta = 0.1$

$\beta = 1$

Final Losses after 10 epochs:

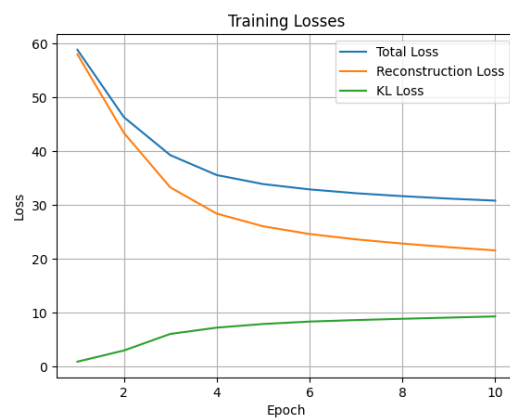
Total Loss: 30.7748

Reconstruction Loss: 21.5248

KL Loss: 9.2500

Epoch	Total Loss	Reconstruction Loss	KL Loss
1	58.815632	57.973191	0.842441
2	46.295754	43.383135	2.912619
3	39.234169	33.240108	5.994061
4	35.532863	28.365359	7.167504
5	33.850414	26.005319	7.845095
6	32.864011	24.569832	8.294179
7	32.150385	23.578825	8.571561
8	31.604948	22.790981	8.813967
9	31.151718	22.119001	9.032717
10	30.774765	21.524800	9.249965

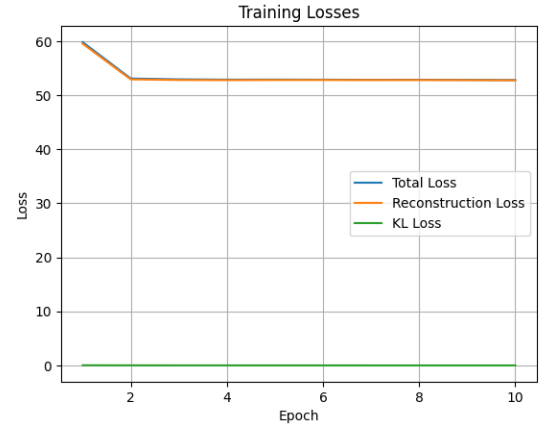
(a) Logs for $\beta = 1$



(b) Graph for $\beta = 1$

$\beta = 10$

```
Final Losses after 10 epochs:
Total Loss: 52.8010
Reconstruction Loss: 52.7507
KL Loss: 0.0050
Epoch Total Loss Reconstruction Loss KL Loss
1 59.813914 59.524812 0.028910
2 53.081924 52.923855 0.015807
3 52.929538 52.812315 0.011722
4 52.875188 52.788368 0.008682
5 52.885318 52.808444 0.007687
6 52.867149 52.809718 0.005743
7 52.847991 52.799477 0.004851
8 52.854635 52.803189 0.005145
9 52.832544 52.781653 0.005089
10 52.800954 52.750701 0.005025
```



(a) Logs for $\beta = 10$

(b) Graph for $\beta = 10$

Figure 7: Comparison of VAE outputs for different values of β .

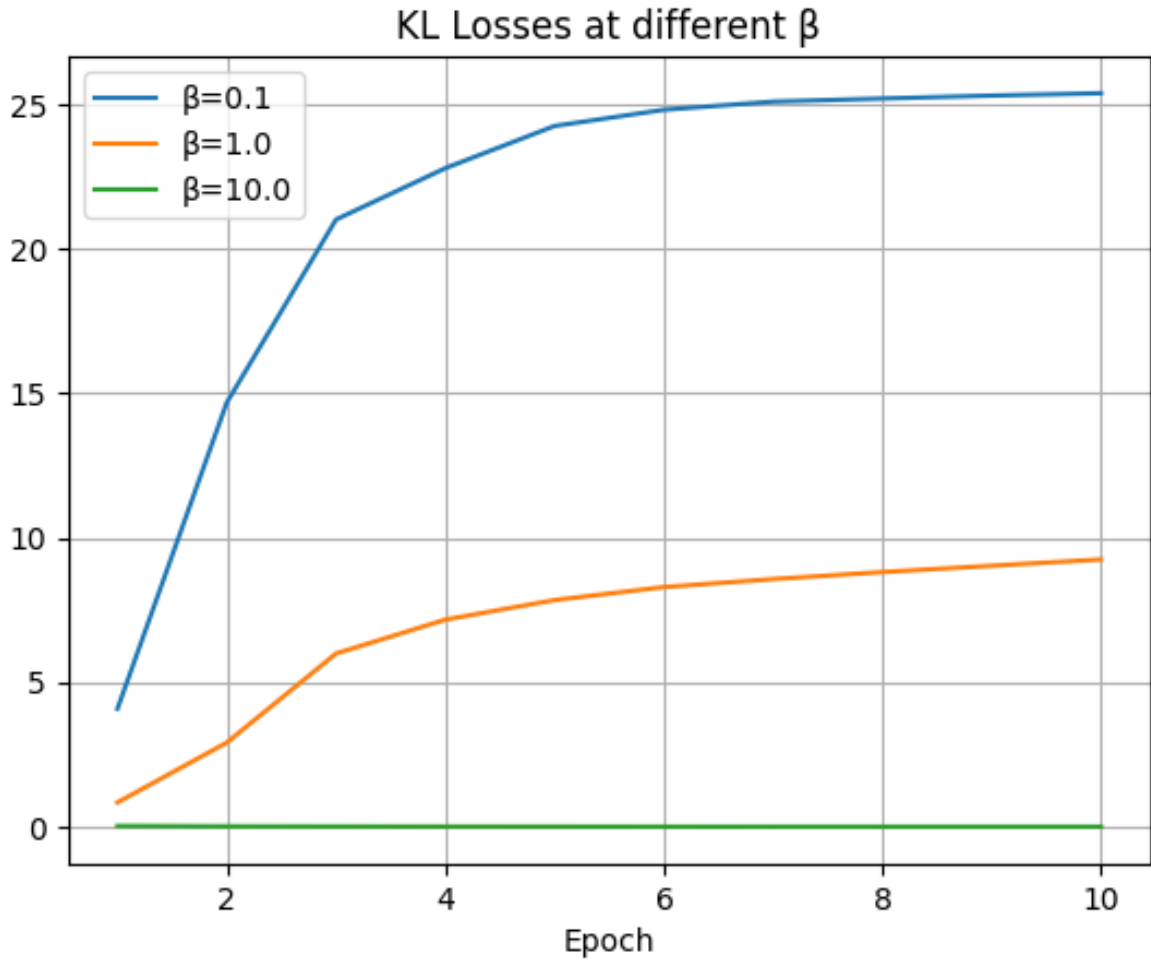


Figure 8: KL loss for different values of β .

3.2 Posterior Collapse Analysis

Here are some of the plots of the per-dimension variance of the means produced by the encoder across the test set.

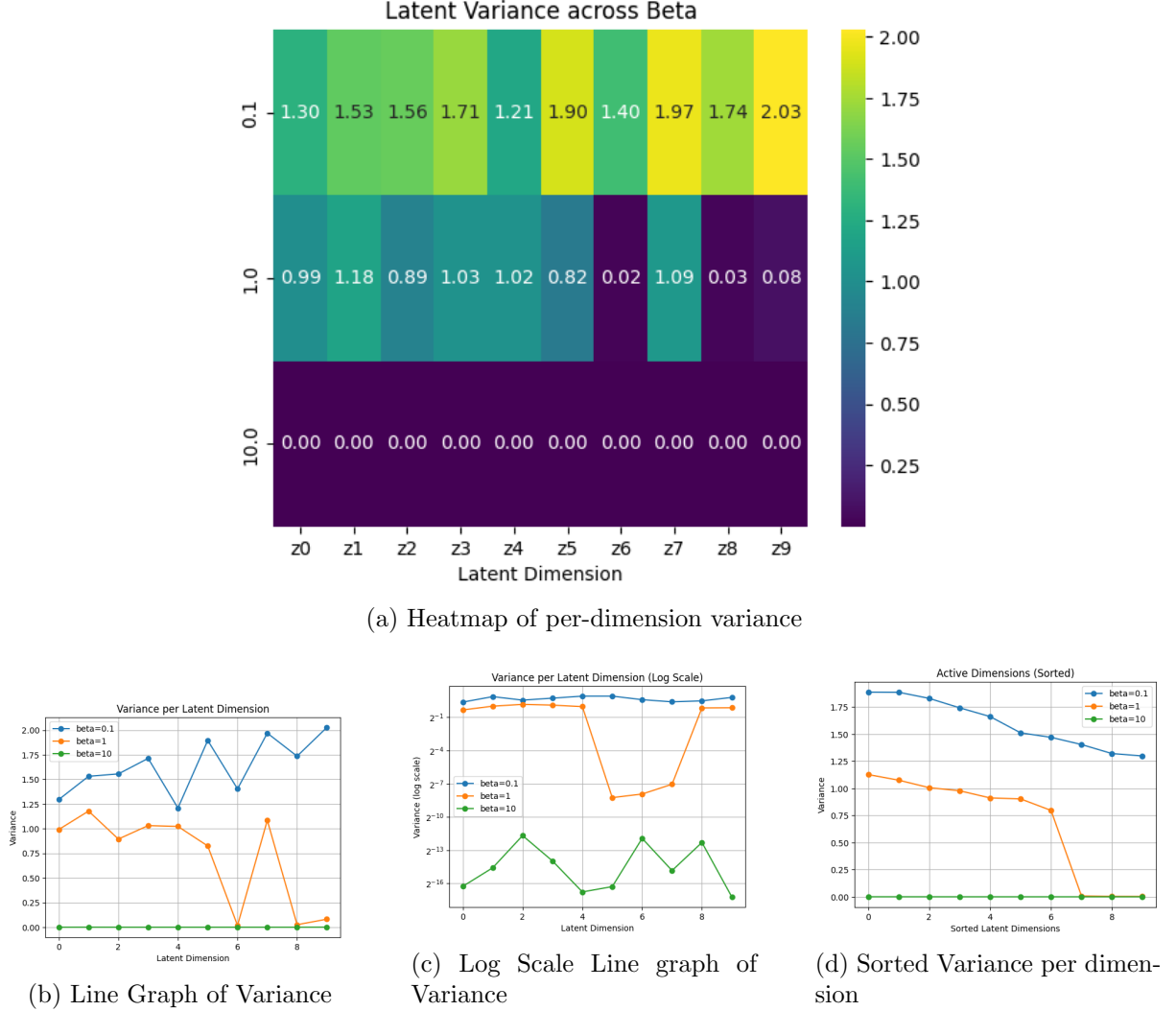


Figure 9: Per-dimension variance of encoder means across the test set. Top: heatmap across all β . Bottom: variance curves for individual β values.

Per-dimension Variance of Encoder Means. High variance indicates an **active** dimension that encodes input-specific information, while near-zero variance indicates a **collapsed** dimension where the encoder outputs values close to the prior mean.

At $\beta = 0.1$, all latent dimensions are active, meaning each dimension encodes information about the input without being strongly constrained by the prior. As a result, when sampling $z \sim p(z)$ at generation time, the sampled points may fall outside the region of latent space that the decoder has learned to map to meaningful outputs, leading to poor generated samples.

For $\beta = 1$, most dimensions remain active. As β increases (e.g., $\beta = 4$), some dimensions begin to collapse. For large values such as $\beta = 10$, most dimensions exhibit near-zero variance, indicating widespread collapse.

Effect of Large β . At $\beta = 10$, almost all the latent dimensions collapse, with $\mu_j \approx 0$ implying $q_\phi(z | x) \approx p(z)$.

This behavior follows from the β -VAE objective:

$$\mathcal{L} = \mathbb{E}[\log p(x | z)] - \beta D_{\text{KL}}(q(z | x) \| p(z)).$$

A large β heavily penalizes the KL divergence, encouraging the encoder to match the prior. As a result, the encoder reduces its dependence on the input, leading to collapse.

Impact on the Decoder. When a latent dimension collapses, the corresponding z_j becomes independent of x and the encoder outputs pure noise sampled from $p(z) = \mathcal{N}(0, 1)$. Consequently, the decoder receives no useful signal from that dimension and learns to ignore it. This creates a feedback loop where unused dimensions remain inactive, resulting in an effectively lower-dimensional latent space and reduced representational capacity. Thus the decoder degenerates into producing a constant output approximating $\mathbb{E}_{p(x)}[x]$, i.e., the dataset mean, regardless of the sampled z . This shows a complete failure of the generative model, as the latent space z becomes entirely uninformative and the VAE loses both its representational and generative capacity.

4 Geometry of the Latent Space

4.1 Linear Interpolation

Here are some of the linear interpolation of 2 and 7 as specified in the question.



Figure 10: Linear interpolations in latent space between digit “2” (x_1) and digit “7” (x_2). Each row shows a different pair of samples, with intermediate images corresponding to equally spaced interpolation steps.

Do we observe a smooth semantic transition? Yes, we get a smooth semantic transition in most cases. The interpolations show a gradual transformation from digit “2” to digit “7” across the interpolation steps, which is consistent with the behavior expected from a well-regularized VAE.

What do the intermediate images look like? The initial steps closely resemble a “2”, gradually losing its characteristic shape as we mix the latent vector of “7”. In the middle steps, the images appear as **ambiguous transitional forms**, sharing features of both digits and often resembling slanted or partially formed strokes. In the later steps, the images progressively resolve into a “7”, acquiring its characteristic horizontal stroke. This demonstrates a smooth semantic interpolation, where structural features of both digits are blended continuously rather than changing abruptly.

Why does KL regularisation enable this? The KL term enforces the approximate posterior to be close to the prior:

$$D_{\text{KL}}(q_{\phi}(z | x) \| p(z)) \approx 0 \Rightarrow q_{\phi}(z | x) \approx \mathcal{N}(0, I).$$

This results in 2 important features of VAE. First, it enforces **continuity** in the latent space, ensuring that points along the interpolation path

$$z_t = (1 - t)\mu_A + t\mu_B$$

lie in regions that the decoder has been trained on. Second, it promotes **overlap between classes**, meaning that representations of different digits *occupy nearby regions in a shared continuous latent space* as they share the same prior. This allows interpolation to remain meaningful throughout.

Why would a standard autoencoder fail? A standard autoencoder lacks the KL regularisation term, allowing the encoder to map inputs to arbitrary regions in latent space. As a result, different classes may form **disconnected clusters**, with large empty regions between them. Interpolating between such clusters leads to latent points that the decoder has never seen during training, producing incoherent outputs. As a result, standard autoencoders do not guarantee smooth or semantically meaningful interpolations.

4.2 2-D Latent Space Visualization

Here is the latent space representation for Z dimension=2 where all μ vectors are colored by the digits.

Class Separation. Some classes are well separated in the latent space. Digit “1” forms a tight and isolated cluster on the far left. Digit “7” occupies a distinct region near the bottom, while digit “0” is broadly distributed on the right but remains clearly identifiable. Digit “3” appears in the upper-centre region with a reasonably compact cluster.

Other classes show moderate separation. Digit “8” overlaps with digits “3” and “5” in the upper-centre region. Digit “6” appears near the centre-right and overlaps with “0” and “4”. Digit “4” lies near the centre and overlaps with “2”, “6”, and “9”. The most significant overlap occurs among digits “2”, “5”, and “9”, which cluster together near the centre of the latent space.

The less separation among the classes mainly occur due to the reduction in the number of latent dimensions. If we increase the number of latent dimensions $K=10$ as earlier, we may see better separation in the 10-dimensional space.

One noticeable thing is that even well separated classes are not islands but are continuous and connected to the other classes due to the shared prior. This lead to the successful linear interpolation in the previous question even though there the number of latent dimensions were 10.

Do visually similar digits cluster together? Yes, strongly. The latent space reflects visual similarity between digits. Digit “1” forms a separate cluster due to its simple vertical stroke. Digits such as “7” and “9” appear near each other due to their similar structure. Human handwriting of these two digits are often very similar. Digits “3”, “5”, and “8” cluster in the upper region of the latent space, sharing curved and loop-like features. Similarly, digits “0” and “6” are located on the right, reflecting their closed or semi-closed loop structures.

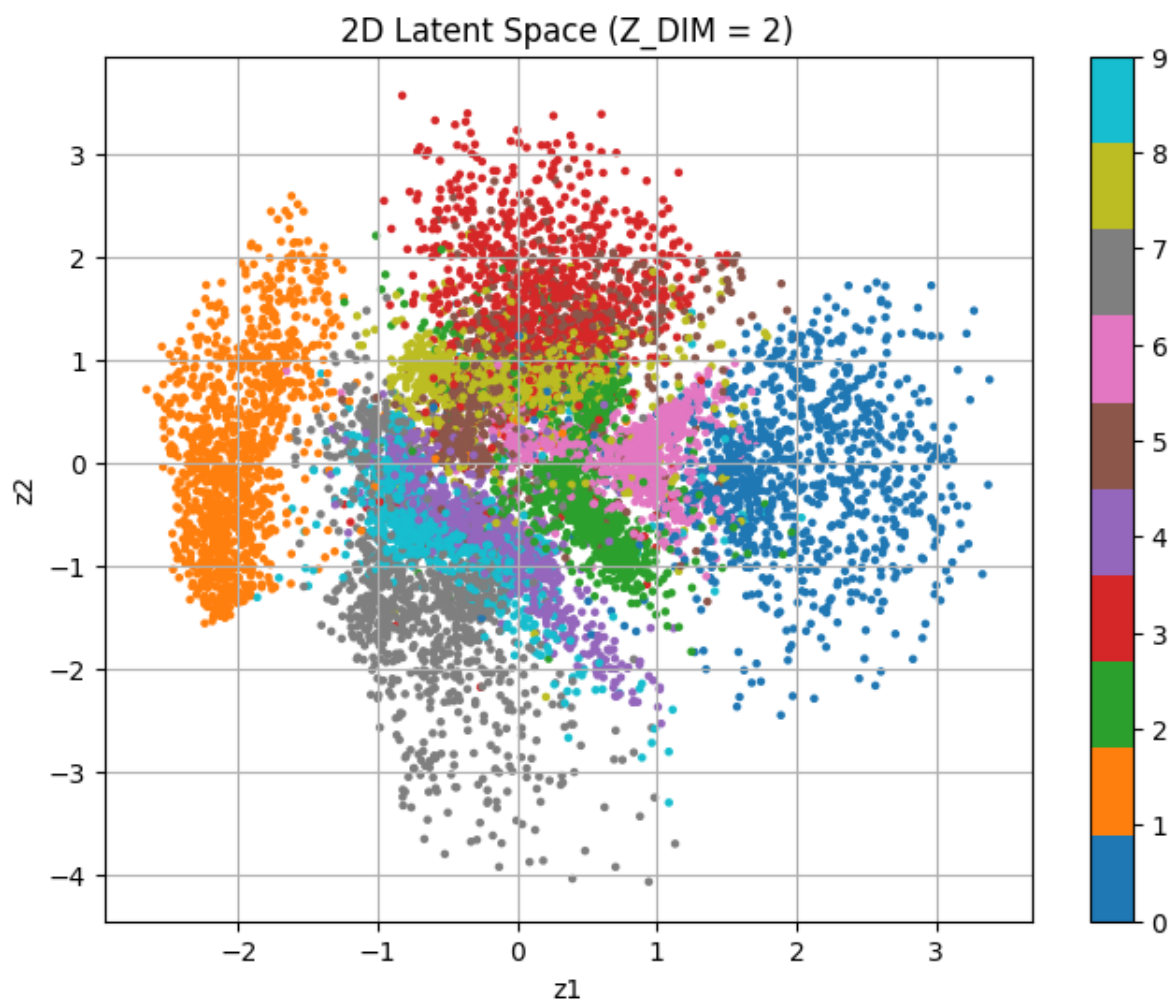


Figure 11: Latent Space Visualization